

11 - Funciones en Javascript

¡Vamos a explorar el concepto de funciones en JavaScript! Las funciones son una de las herramientas más importantes y poderosas del lenguaje. Nos permiten organizar nuestro código, hacerlo más legible y reutilizable. Además, nos ayudan a evitar repetir bloques de código una y otra vez.

¿Qué es una función en JavaScript?

Una función en JavaScript es un bloque de código reutilizable que realiza una tarea específica. Puedes pensar en una función como una "máquina" que toma entradas (opcionalmente), hace algo con esas entradas y luego devuelve un resultado o simplemente ejecuta una acción.

Sintaxis básica de una función

Para declarar una función básica en JavaScript, usamos la palabra clave `function`. Aquí tienes la estructura básica de una función:

```
function nombreDeLaFuncion(parámetros) {  
  // Bloque de código que se ejecuta  
  return valor; // Opcional: devuelve un valor  
}
```

- **nombreDeLaFuncion:** El nombre que le das a la función para poder llamarla más tarde.
- **parámetros:** Son las entradas que le pasas a la función (opcional).
- **return:** La función puede devolver un valor (esto también es opcional).

Ejemplo 1: Función simple

```
function saludar(nombre) {  
  return "Hola, " + nombre + "!";  
}  
  
console.log(saludar("Carlos")); // Imprime "Hola, Carlos!"
```

Explicación:

La función saludar toma un parámetro nombre, lo concatena con "Hola, " y devuelve la frase completa. Cuando llamamos a saludar("Carlos"), la salida será "Hola, Carlos!".

Ejemplo 2: Función que no devuelve nada

```
function mostrarMensaje() {  
  console.log("Este es un mensaje simple.");  
}  
  
mostrarMensaje(); // Imprime "Este es un mensaje simple."
```

Explicación:

Aquí, la función mostrarMensaje no toma parámetros ni devuelve un valor. Solo imprime un mensaje cuando es llamada.

Beneficios de usar funciones

Reutilización de código: En lugar de escribir el mismo bloque de código varias veces, puedes usar una función para evitar la repetición. Solo necesitas llamarla cuando la necesites.

Modularidad: Las funciones permiten dividir el código en pequeñas secciones que hacen tareas específicas, lo que facilita la lectura y el mantenimiento.

Mantenibilidad: Si tienes un cambio en una tarea común, solo necesitas actualizar la función, y todas las llamadas a esa función se actualizarán automáticamente.

Flexibilidad: Puedes pasar diferentes valores a una función y obtener resultados personalizados.

Funciones anónimas

Una función anónima es una función que no tiene nombre. Generalmente, se utilizan cuando necesitas crear una función de manera rápida y no necesitas reutilizarla más tarde. Las funciones anónimas son muy útiles cuando se pasan como argumentos a otras funciones.

Ejemplo: Función anónima asignada a una variable

```
let sumar = function(a, b) {  
  return a + b;  
};  
  
console.log(sumar(3, 4)); // Imprime 7
```

Explicación:

Aquí, hemos creado una función anónima y la hemos asignado a la variable `sumar`. Luego llamamos a `sumar(3, 4)`, que devuelve 7.

Las funciones anónimas son esenciales en situaciones donde solo quieres definir una función en el momento, sin la necesidad de reutilizarla en otros lugares del código.

Funciones flecha (arrow functions)

Las funciones flecha fueron introducidas en ES6 y son una forma más concisa de escribir funciones. Tienen una sintaxis más corta y, además, tienen un comportamiento especial con el contexto de `this` (aunque no vamos a profundizar en eso ahora).

Sintaxis de una función flecha

```
const nombreDeLaFuncion = (parámetros) => {  
  // Bloque de código  
  return valor;  
};
```

Si la función solo tiene una línea de código y devuelve algo, podemos hacerla aún más concisa omitiendo las llaves `{}` y el `return`.

Ejemplo 1: Función flecha simple

```
const multiplicar = (a, b) => a * b;  
  
console.log(multiplicar(5, 4)); // Imprime 20
```

Explicación:

En este ejemplo, la función flecha `multiplicar` toma dos parámetros `a` y `b` y devuelve su producto. Como solo tiene una línea de código, no necesitamos las llaves ni la palabra `return`.

Ejemplo 2: Función flecha con múltiples líneas

```
const saludar = (nombre) => {  
  let mensaje = "Hola, " + nombre + "!";  
  return mensaje;  
};
```

```
};  
  
console.log(saludar("María")); // Imprime "Hola, María!"
```

Explicación:

Cuando una función tiene más de una línea de código, debemos usar las llaves {} y la palabra clave return si queremos devolver un valor.

Beneficios de las funciones flecha:

- Sintaxis más concisa: Reducen la cantidad de código necesario para declarar una función.
- No modifican el valor de this: Esto puede ser útil en ciertos contextos, como cuando trabajas dentro de métodos o callbacks. (Veremos más sobre this más adelante).

Ejemplos de uso de funciones

Ejemplo 1: Función que calcula el área de un triángulo

```
function areaTriangulo(base, altura) {  
  return (base * altura) / 2;  
}  
  
console.log(areaTriangulo(10, 5)); // Imprime 25
```

Explicación:

Esta función toma dos parámetros (base y altura) y devuelve el área del triángulo utilizando la fórmula $(base * altura) / 2$.

Ejemplo 2: Función flecha para verificar si un número es par

```
const esPar = (numero) => numero % 2 === 0;

console.log(esPar(10)); // Imprime true
console.log(esPar(7));  // Imprime false
```

Explicación:

Aquí, la función flecha esPar devuelve true si el número es divisible por 2, y false si no lo es.

Ejercicio: Crear una calculadora simple

Ejercicio:

Crea una serie de funciones que realicen operaciones matemáticas básicas: suma, resta, multiplicación y división. Cada una de estas funciones debe tomar dos parámetros (los números a operar) y devolver el resultado.

Código

```
function sumar(a, b) {
  return a + b;
}

function restar(a, b) {
  return a - b;
}

function multiplicar(a, b) {
  return a * b;
}

function dividir(a, b) {
  if (b !== 0) {
    return a / b;
  } else {
    return "No se puede dividir por cero";
  }
}
```

```
console.log(sumar(5, 3));           // Imprime 8
console.log(restar(10, 4));         // Imprime 6
console.log(multiplicar(7, 3));     // Imprime 21
console.log(dividir(8, 2));         // Imprime 4
console.log(dividir(8, 0));         // Imprime "No se puede dividir
por cero"
```

Conclusión

Las funciones en JavaScript son una herramienta increíblemente poderosa que te permite organizar tu código, hacerlo más limpio y reutilizable. Puedes crear funciones con nombre, funciones anónimas o usar la sintaxis moderna de funciones flecha. A medida que programes más, notarás lo útiles que son para simplificar y modularizar el código. ¡Practicar con ejemplos te ayudará a dominar el uso de funciones en poco tiempo!

EJERCICIOS:

Ejercicio 1: Función para verificar si un número es par o impar

Crea una función llamada `esPar` que tome un número como parámetro y devuelva `true` si es par, o `false` si es impar.

Pista: Un número es par si es divisible por 2.

```
function esPar(numero) {
  // Completa el código aquí
}

// Llamadas de ejemplo
console.log(esPar(4)); // true
console.log(esPar(7)); // false
```

Ejercicio 2: Calculadora básica

Crea cuatro funciones llamadas sumar, restar, multiplicar y dividir. Cada una debe tomar dos números como parámetros y devolver el resultado de la operación correspondiente.

```
function sumar(a, b) {  
    // Completa el código aquí  
}  
  
function restar(a, b) {  
    // Completa el código aquí  
}  
  
function multiplicar(a, b) {  
    // Completa el código aquí  
}  
  
function dividir(a, b) {  
    // Completa el código aquí  
}  
  
// Llamadas de ejemplo  
console.log(sumar(5, 3));           // 8  
console.log(restar(10, 7));         // 3  
console.log(multiplicar(4, 6));     // 24  
console.log(dividir(12, 4));        // 3
```

Ejercicio 3: Conversor de temperatura

Crea una función llamada convertirACelsius que tome una temperatura en grados Fahrenheit como parámetro y devuelva la temperatura en grados Celsius. La fórmula para convertir Fahrenheit a Celsius es:

$$C = (F - 32) \times 5/9$$

$$C = (F - 32) \times 9/5$$


```
function convertirACelsius(fahrenheit) {  
    // Completa el código aquí  
}  
  
// Llamada de ejemplo  
console.log(convertirACelsius(100)); // 37.78
```

Ejercicio 4: Función de mayor de dos números

Crea una función llamada mayor que tome dos números como parámetros y devuelva el mayor de los dos. Si los números son iguales, puede devolver cualquiera de los dos.

```
function mayor(a, b) {  
    // Completa el código aquí  
}  
  
// Llamadas de ejemplo  
console.log(mayor(10, 20)); // 20  
console.log(mayor(15, 15)); // 15
```

Ejercicio 5: Suma de números en un rango

Crea una función llamada sumaRango que tome dos números (un inicio y un fin) como parámetros y devuelva la suma de todos los números entre esos dos valores, incluyendo el inicio y el fin.

```
function sumaRango(inicio, fin) {  
    // Completa el código aquí  
}  
  
// Llamada de ejemplo  
console.log(sumaRango(1, 5)); // 15 (1 + 2 + 3 + 4 + 5)
```

Ejercicio 6: Función flecha para calcular el área de un triángulo

Escribe una función flecha que calcule el área de un triángulo. La fórmula para calcular el área de un triángulo es:

$$A = \text{base} \times \text{altura} / 2$$

$$A = 2 \times \text{base} \times \text{altura}$$

```
const areaTriangulo = (base, altura) => {  
  // Completa el código aquí  
};  
  
// Llamada de ejemplo  
console.log(areaTriangulo(5, 10)); // 25
```

Ejercicio 7: Factorial de un número

Crea una función llamada factorial que tome un número como parámetro y devuelva su factorial. El factorial de un número es el producto de todos los números desde 1 hasta ese número. Por ejemplo, el factorial de 5 es:

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

```
function factorial(numero) {  
  // Completa el código aquí  
}  
  
// Llamadas de ejemplo  
console.log(factorial(5)); // 120  
console.log(factorial(3)); // 6
```

Ejercicio 8: Función para contar vocales

Escribe una función llamada contarVocales que tome una cadena de texto como parámetro y devuelva el número de vocales que contiene.

```
function contarVocales(texto) {  
  // Completa el código aquí  
}  
  
// Llamada de ejemplo  
console.log(contarVocales("JavaScript")); // 3
```

Ejercicio 9: Promedio de un array

Crea una función llamada promedio que tome un array de números como parámetro y devuelva el promedio de esos números.

```
function promedio(numeros) {  
  // Completa el código aquí  
}  
  
// Llamada de ejemplo  
console.log(promedio([10, 20, 30, 40])); // 25
```

Ejercicio 10: Convertir a mayúsculas

Crea una función llamada convertirMayusculas que tome una cadena de texto como parámetro y devuelva la misma cadena, pero con todas las letras en mayúsculas.

```
function convertirMayusculas(texto) {
```

```
// Completa el código aquí  
}  
  
// Llamada de ejemplo  
console.log(convertirMayusculas("javascript es genial")); //  
"JAVASCRIPT ES GENIAL"
```

Estos ejercicios son una excelente manera de reforzar los conceptos de funciones, trabajar con diferentes tipos de entradas y desarrollar una comprensión sólida de cómo funcionan las funciones en JavaScript. ¡Disfruta resolviéndolos!